# ProFolio AI

Intelligent Portfolio Management &
Job-Specific Resume System

Project Report | Author: Jui Tawde

## At a glance

| | |
|---|---|
| Repository | github.com/juitawde/resume-builder |
| Local app | http://localhost:5180 |
| Frontend | React 18, Vite, Vanilla CSS, Lucide React |
| Backend | Node.js, Express.js (ES Modules), MongoDB, Mongoose |
| AI | Google Gemini API |
| Exports | PDF, DOCX, XLSX, Google Sheets |

**ProFolio AI** is a full-stack application that helps users organize their professional information in one centralized portfolio and generate resumes tailored to specific job descriptions. Instead of rewriting a resume for every application, users maintain a structured database of skills, education, experience, projects, certifications, and achievements. AI-assisted analysis identifies relevant information and produces job-specific resumes as downloadable PDF and Word files, with Google Sheets integration for synchronization and exports.

> Central idea: maintain professional information once, then reuse it intelligently for different career opportunities.

# 1. Key Features

## Centralized Professional Portfolio

A structured portfolio organized into these categories:

| Category | Contents |
|---|---|
| Personal Information | Name, email, phone, location, title, LinkedIn, GitHub, website |
| Professional Summary | Career objectives and professional introduction |
| Education | Institutions, degrees, fields, dates, grades, honors |
| Skills | Technical and soft skills by category (frontend, backend, databases, cloud, AI/ML, languages, tools) |
| Experience | Employment, internships, responsibilities, tech stacks, measurable achievements |
| Projects | Descriptions, repositories, live links, technologies, highlights |
| Certifications | Courses, credentials, verification links, IDs |
| Achievements | Awards, hackathons, leadership, other accomplishments |
| Research and Publications | Papers, abstracts, publication details, DOI links |
| Languages and Competencies | Language proficiency and additional competencies |

## AI-Powered Portfolio Parsing

Gemini converts unstructured professional information into structured data:

- Extracts information from raw biographies and summaries.
- Organizes it into the relevant portfolio categories.
- Reduces manual data entry and feeds later resume generation.

> AI-generated content should always be reviewed before being used in a real job application.

## Job Description Analysis

Identifies required technical skills, relevant technologies and tools, job-specific keywords, core responsibilities and qualifications, and where the portfolio matches or falls short of the role.

## Job-Specific Resume Generation

1. Enter or review the target job description.
2. Identify relevant skills, projects, and experiences.
3. Select what should appear in the resume.
4. Generate a tailored summary and experience descriptions.
5. Review through a live preview.
6. Export the final document.

### Accuracy-Focused AI Assistance

Generation is grounded in portfolio data. Safeguards are designed to:

- Preserve original achievements and numerical metrics.
- Avoid inventing qualifications or experience.
- Highlight missing skills instead of falsely claiming proficiency.
- Improve bullets using relevant action verbs.
- Align with the job description without misrepresenting the candidate.

### Interactive Resume Tailoring Workspace

A split-screen workflow with a job description input area, interactive resume preview, selection controls for skills, projects and experience, content customization, export actions, and access to previously generated resumes.

### PDF and Word Export

| Format | Purpose |
|---|---|
| PDF | Sharing, uploading to job portals, consistent layout |
| DOCX | Editable Word document to review and modify before submission |

### Saved Resume History

Keep multiple resume versions from the same portfolio. A full-stack role and an AI engineering role can emphasize different content from the same underlying data.

### Google Sheets Integration

- **Apps Script webhook:** sync through a deployed web app.
- **Google Sheets API:** direct updates using service-account credentials.
- **Excel export:** an .xlsx workbook with eight worksheets.

### Modern User Interface

React and Vite with a custom CSS design system: DM Sans typography, a soft rose and neutral palette, modular components, and Lucide icons.

## 2. Technology Stack

| Layer | Technologies |
|---|---|
| Frontend | React 18, JavaScript, Vite |
| UI and styling | Vanilla CSS, CSS custom properties |
| Icons | Lucide React |
| Backend | Node.js, Express.js |
| Module system | ES Modules |
| Database | MongoDB |

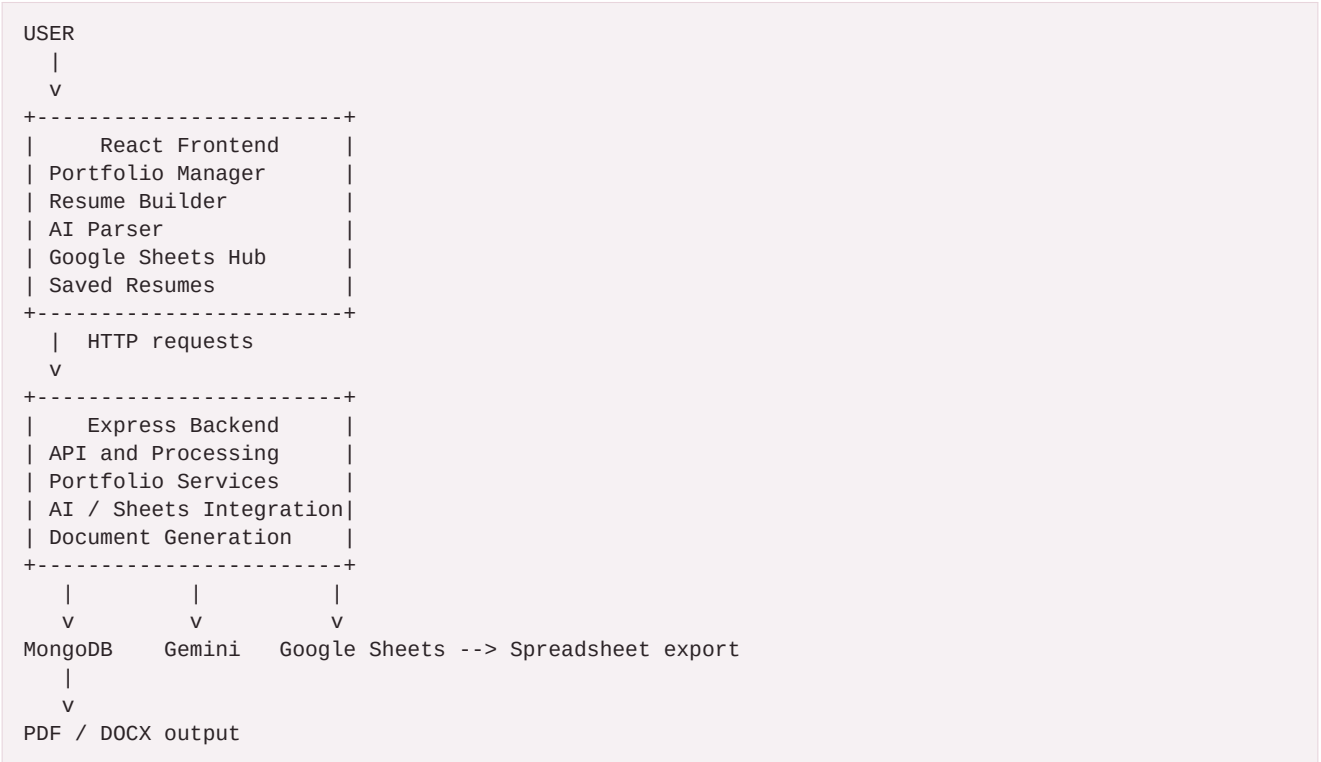
| Layer | Technologies |
|---|---|
| Object modeling | Mongoose |
| AI integration | Google Gemini API |
| Word generation | docx |
| Spreadsheets | xlsx |
| Google integrations | Google APIs, Google Apps Script |
| Configuration | dotenv |
| Cross-origin | CORS |

# 3. Screenshots

- Screenshot 1
- Screenshot 2
- Screenshot 3
- Screenshot 4

# 4. Application Architecture

A client-server architecture. The React frontend handles portfolio editing, job description input, customization and preview. The Express backend handles data operations, AI integration, spreadsheet services and document generation.




**Data flow:** users manage information in the frontend; the backend stores it in MongoDB, calls configured external services, and generates downloadable documents. Exact request flow varies by feature.

# 5. Project Structure

```
resume-builder/
|-- client/
|   |-- src/
|   |   |-- App.jsx
|   |   |-- index.css
|   |   `-- components/
|   |       |-- AIParserModal.jsx
|   |       |-- GoogleSheetsModal.jsx
|   |       |-- PortfolioManager.jsx
|   |       |-- ResumeBuilder.jsx
|   |       |-- SavedResumes.jsx
|   |       |-- Sidebar.jsx
|   |       `-- TopHeader.jsx
|   |-- vite.config.js
|   `-- package.json
|-- server/
|   |-- server.js
|   |-- services/
|   |   |-- docxService.js
|   |   |-- sheetsService.js
|   |   `-- ...
|   `-- package.json
|-- package.json
`-- README.md
```

## Important Frontend Files

| File | Responsibility |
|---|---|
| App.jsx | Main application component and interface composition |
| index.css | Global styles, typography, colors, design-system rules |
| AIParserModal.jsx | AI-assisted parsing interface |
| GoogleSheetsModal.jsx | Google Sheets actions |
| PortfolioManager.jsx | Portfolio management interface |
| ResumeBuilder.jsx | Resume customization and generation workspace |
| SavedResumes.jsx | Review saved resume versions |
| Sidebar.jsx | Sidebar navigation |
| TopHeader.jsx | Header and top-level controls |
| vite.config.js | Vite dev and build configuration |
| package.json | Frontend dependencies and scripts |

## Important Backend Files

| File | Responsibility |
|---|---|
| server.js | Express entry point |
| services/docxService.js | Word document generation |
| services/sheetsService.js | Google Sheets integration and spreadsheet operations |
| package.json | Dependencies, module config, scripts |

# 6. Prerequisites

- Node.js 18 or newer (Node.js 24 documented as a tested environment).
- npm.
- MongoDB locally, or a MongoDB Atlas connection string.
- A Google Gemini API key.
- Google Sheets configuration, if synchronization is needed.

```
node -v
npm -v
```

# 7. Installation and Setup

## Step 1: Clone the repository

```
git clone https://github.com/juitawde/resume-builder.git
cd resume-builder
```

Repository: https://github.com/juitawde/resume-builder

## Step 2: Install dependencies

```
npm run install:all

# or separately
cd server && npm install
cd ../client && npm install
```

## Step 3: Configure environment variables

Create `server/.env` (copy `.env.example` if present):

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/portfolio_builder
GEMINI_API_KEY=your_gemini_api_key_here
GOOGLE_APPS_SCRIPT_WEBHOOK_URL=
GOOGLE_SERVICE_ACCOUNT_EMAIL=
GOOGLE_PRIVATE_KEY=
GOOGLE_SPREADSHEET_ID=
```

| Variable | Purpose |
|---|---|
| PORT | Backend server port |
| MONGODB_URI | MongoDB connection string |
| GEMINI_API_KEY | Key for Gemini-powered features |
| GOOGLE_APPS_SCRIPT_WEBHOOK_URL | Deployed Apps Script web app URL |
| GOOGLE_SERVICE_ACCOUNT_EMAIL | Service-account email for direct API use |
| GOOGLE_PRIVATE_KEY | Private key of the service account |

| Variable | Purpose |
| --- | --- |
| GOOGLE_SPREADSHEET_ID | ID of the target spreadsheet |

> Never commit real credentials. The Apps Script webhook and direct Sheets API are alternative paths; configure only the one you intend to use.

### Step 4: Start MongoDB

Ensure the local service is running (default `mongodb://127.0.0.1:27017/portfolio_builder`). For Atlas, use your connection string and confirm the database user permissions and network access rules.

### Step 5: Start the backend (port 5000)

```
npm run server

# or
cd server && npm run dev
```

### Step 6: Start the frontend (port 5180)

```
npm run client

# or
cd client && npm run dev
```

Open http://localhost:5180 and keep both servers running.

## 8. Gemini API Configuration

1. Visit Google AI Studio.
2. Obtain a Gemini API key.
3. Add `GEMINI_API_KEY` to `server/.env`.
4. Restart the backend.

Without a key, the design falls back to heuristic matching. This is useful for testing the interface and matching flow but is not equivalent to AI-powered generation.

## 9. Google Sheets Integration

### Option A: Apps Script webhook

1. Create a spreadsheet at sheets.new.
2. Open Extensions, then Apps Script.
3. Use the integration code from the project's Sheets service and save.
4. Deploy as a web app and set access per your security needs.
5. Copy the deployment URL into `GOOGLE_APPS_SCRIPT_WEBHOOK_URL`.
6. Use the app's synchronization workflow.

Treat the webhook URL as a sensitive endpoint and review what data it accepts and who can call it.

### Option B: Google Sheets API

```
GOOGLE_SERVICE_ACCOUNT_EMAIL=your_service_account_email
GOOGLE_PRIVATE_KEY=your_service_account_private_key
GOOGLE_SPREADSHEET_ID=your_spreadsheet_id
```

Enable the Google API, configure the service account, and share the target spreadsheet with it. Preserve newline characters in the private key when using a hosting provider.

### Export to Excel

A multi-worksheet .xlsx export (eight worksheets) lets you open data in Excel or import it into Google Sheets without a live connection.

## 10. How to Use ProFolio AI

1. **Build your portfolio.** Add personal details, summary, education, skills, experience, projects, certifications and achievements.
2. **Import existing information.** Use AI parsing to structure raw text, then review and correct it.
3. **Select a target job.** Paste the job description.
4. **Analyze requirements.** Review skills and keywords, and spot gaps.
5. **Customize the resume.** Choose relevant skills, projects and experience; verify generated text.
6. **Preview and export.** Export PDF or DOCX; check formatting, contacts, dates, spelling and facts.
7. **Manage versions.** Keep a separate resume per role.

## 11. Example Use Cases

| Role | Emphasis |
|---|---|
| Senior Full Stack Cloud Engineer | React and frontend; Node.js and Express; MongoDB and APIs; cloud and AWS; microservices and scalable architecture |
| Staff AI / Generative AI Engineer | Generative AI and LLMs; Gemini API; RAG; vector databases; Python; AI pipeline development and reliability |

One portfolio supports different application strategies without maintaining separate copies of the candidate's history. Content is prioritized only when supported by the candidate's actual information.

## 12. Build and Development Commands

| Command | Purpose |
|---|---|
| npm run install:all | Install frontend and backend dependencies from the root |
| npm run server | Start the backend dev server |
| npm run client | Start the Vite frontend dev server |
| npm run dev | Show instructions for running both in separate terminals |
| npm run build | Production frontend build (inside client/) |

| Command | Purpose |
|---|---|
| npm run preview | Preview the built frontend (inside client/) |
| npm start | Start the backend directly (inside server/) |

The build outputs to Vite's default `dist/` directory unless configured otherwise.

# 13. Security Considerations

- Keep .env files out of version control; never publish API keys or private keys.
- Use environment variables for secrets.
- Restrict database access to authorized users and services.
- Grant Google service accounts only the permissions they need.
- Validate incoming requests on the backend.
- Review access permissions on deployed Apps Script webhooks.
- Avoid storing unnecessary sensitive personal information.
- Review AI-generated content before professional use.
- Rotate credentials immediately if exposed.

> Removing a secret from the latest commit does not remove it from Git history. If a real credential was exposed, revoke or rotate it.

# 14. Troubleshooting

| Problem | What to check |
|---|---|
| Frontend will not start | Run npm install in client/, then npm run dev. If port 5180 is busy, free it or configure another. |
| Backend will not start | Run npm install in server/; check the terminal for missing dependencies, syntax and environment errors. |
| MongoDB connection fails | MongoDB running, MONGODB_URI correct, credentials valid, Atlas network rules allow access. |
| AI unavailable | GEMINI_API_KEY set and valid in server/.env; restart the backend. |
| Sheets sync fails | Integration method, webhook URL or credentials, spreadsheet ID, permissions, API configuration. |
| Export fails | Backend logs, document-generation dependencies installed, input data and returned errors. |

# 15. Potential Future Improvements

- More customizable resume layouts and themes.
- Templates optimized for different industries.
- More detailed job-description match reports.

- Improved accessibility and mobile responsiveness.
- Stronger validation of imported portfolio data.
- Automated tests for resume generation and Sheets sync.
- Granular privacy and sharing controls.
- Better error handling for external API failures.
- Deployment documentation and production configuration.
- Job application tracking alongside saved resumes.

*These are potential improvements, not claims that the features already exist.*

## 16. Project Goals and Author

ProFolio AI aims to make portfolio management and job-specific resume generation more efficient by combining structured data storage, AI-assisted writing, document generation and spreadsheet integration. It brings together full-stack development, database modeling, external API integration, AI-assisted content generation and document automation.

**Author:** Jui Tawde | GitHub repository